OPEN SOURCE HARDWARE · REVISION v4

HOS Trigonometry 1.5

Humanoid Operating System
Hardware documentation for OSHWA certification

Prolabs Robotics · Aarav Jaisingh · Siyona Chicker
github.com/Aarav-111/Humanoid-Operating-System
CC BY-SA 4.0 documentation · CERN-OHL-S-2.0 hardware

00 READ FIRST — System requirements (Revision v4)

HOS Trigonometry 1.5 requires a laptop (or desktop computer) to run.

The project's Python software — **S1.5-SRC / HOS-X (the laptop-side .py file)** — runs **on the laptop, not on the Arduino**. It provides the GUI, the camera input and AprilTag navigation, FastSAM object outlining, the OpenAI API calls (planning, object naming, error checking, speech-to-text) and task assignment.

The **Arduino Uno is the only microcontroller**. It runs **only the firmware shell** (Hardware/Firmware/Shell_1.5.ino). It cannot run Python or make API calls; it receives short serial commands from the laptop over USB (115200 baud) and drives the stepper motors and the gripper, and reads the joystick, IR and time-of-flight sensors.

ELEMENT	RUNS	ROLE
Laptop (required) + overhead USB webcam	Python software (S1.5-SRC / HOS-X)	GUI, vision, planning, API calls, sends encoded serial commands
USB cable	—	Serial link laptop ↔ Uno, and Uno logic power
Arduino Uno + CNC Shield V3 + 4× A4988	Firmware shell (Shell_1.5.ino) only	Step pulses, gripper, sensors
4× NEMA 17, 12 V 10 A supply, sensors, frame, printed parts	—	The arm

What changed in this revision. In response to OSHWA review: this page was added; former Section 14 ("What to publish for certification") was removed and later sections renumbered (Safety is now 14, Certification checklist is now 15); Sections 16–22 were added with the laptop requirements, a complete bill of materials including the laptop, wiring for the shipped firmware, step-by-step assembly, software setup, first power-on test and troubleshooting. Sections 1–13 are unchanged. Where Section 9 (BOM) or Section 10 (pin map) differ from Sections 17–18, Sections 17–18 take precedence. Values the builders have not yet confirmed are marked TODO: confirm.

Contents of the added material: 16 Laptop & system overview · 17 Complete bill of materials · 18 Wiring · 19 Mechanical assembly · 20 Software setup · 21 First power-on test & troubleshooting · 22 Additional safety notes.

HARDWARE DOCUMENTATION FOR OSHWA CERTIFICATION

HOS Trigonometry 1.5

> Humanoid Operating System · robotic-arm Cartesian layer only · physical joystick command
> Arduino Uno · CNC Shield V3 · A4988 drivers · analog joystick · NEMA 17 steppers · time-of-flight · IR grip sensor

Name: HOS Trigonometry 1.5. Version: 1.5. Scope: the robotic-arm Cartesian layer only. It is not the mobile base, not the vision stack, and not Shell 1.5. It belongs to the Humanoid Operating System published at github.com/Aarav-111/Humanoid-Operating-System by Prolabs Robotics (project lead Aarav Jaisingh, co-lead Siyona Chicker). HOS splits a robotic arm (the worker) from a mobile base (the mover). Trigonometry 1.5 is only the arm: a three-axis Cartesian mechanism that carries the tool through X, Y and Z.

This file is the hardware document for that version. It is written so another builder can study, modify, and reproduce the arm. It is not itself an OSHWA certificate. Certification is a separate submission at certification.oshwa.org once this document and the design files are public.

What is original, and what is not. Arduino Uno, CNC Shield V3, A4988 modules, the analog joystick breakout, NEMA 17 motors, the metal 10 A supply, belts, pulleys, and the leadscrew are third-party modules with public datasheets. The creator contribution is the Cartesian integration, the lid-mounted physical command console, the hybrid belt/leadscrew drive, and Trigonometry 1.5, the firmware that resolves a joystick vector or an HOS grid cell into step pulses.

01 What HOS Trigonometry 1.5 is

HOS separates a robotic arm from a mobile base. This document covers the arm only. Trigonometry 1.5 is that arm in Cartesian form: belts on X and Y, a leadscrew on Z, a tool motor on the carriage, and a joystick that jogs those three axes. Locomotion, navigation, and the camera planner are other HOS layers and are out of scope here.

X and Y are belt-driven on aluminium V-slot. Z is a leadscrew with parallel smooth-rod guidance. Motion electronics are an Arduino Uno, a CNC Shield V3, and A4988 drivers, fed by a separate 10 A metal supply. An HOS plan can ask this arm for a grid move — `goto_coordinate(T,8)`, `pickup()`, `keep()` — but the trigonometry that executes it lives on this stage: column and row become millimetres, a joystick deflection becomes a vector, and the vector becomes STEP/DIR on the arm axes only. Two sensors close that loop on the arm itself. A time-of-flight sensor on the carriage reads distance to the ground, so Z is a measured height, not only a step count. IR sensors are mounted horizontally across the jaw gap, so the beam sees an object before the sponge closes on it.

The operator does not have to sit at a laptop to move the arm. The analog joystick on the controller lid is the hand control for teach and jog of X, Y and Z.

02 What is different

A stock GRBL plotter or mini-mill is a PC peripheral. This build adds a command layer that lives on the machine.

Physical software. The joystick is not a decoration. Its two potentiometers and switch are read by the Arduino. Deflection becomes a signed step rate on X and Y; the switch selects Z, enables the drivers, or

latches a mode. The same STEP/DIR pins the shield already wires to the A4988s carry those pulses. One firmware owns both the human command and the motor timing. That is the "physical software command" layer: the control law is software, the affordance is a hand control.

Hybrid axes, on purpose. Belts on X and Y give speed and low moving mass for jogging across the table. The Z leadscrew gives holding torque and finer pitch when the tool is loaded vertically. A pure-belt Z would back-drive. A pure-screw XY would be slow to teach. The split matches the job.

Console on the power brick. The controller is not a loose breadboard. The Uno and shield sit in a white printed (or foam-board) bay, the joystick is fixed to the lid, and that bay sits on the metal supply. Power, logic, and the hand control are one station at the corner of the frame.

A tool motor on the carriage. A horizontal NEMA 17 on the Z carriage, separate from the vertical leadscrew motor, carries the end effector. That actuator is part of the arm, not a fourth positioning axis. The shield's A socket drives it.

Ground distance and grip presence. A time-of-flight sensor is aimed down from the carriage and reports millimetres to the ground or table. That reading is the arm's Z truth: jog and grid moves stop at a height floor instead of trusting the leadscrew count alone. An IR sensor is mounted horizontally at the gripper, beam across the jaw gap, and returns present or absent. The jaw face is a sponge, for a compliant grip. `pickup()` is allowed only when the horizontal IR sees an object. An empty close is refused.

03 What is well made

The frame is commodity 2020-class V-slot, joined by white printed brackets rather than machined plates. That is the right call for a reproducible student or farm-lab machine: a builder with a printer and a hacksaw can copy the nodes. Motor mounts, belt anchors, and the Z upright are the same family of parts, so a broken bracket is a reprint, not a shop job.

Motor power and logic power are split. The metal supply (label: 10 A, Keenonic-class enclosed SMPS) feeds the shield's VMOT. The Uno is powered over USB. A4988 modules are socketed, so a cooked driver is a swap. Z is constrained by the screw and two smooth rods, which keeps the carriage from twisting when the horizontal tool motor and its cable weight hang off the front.

Cable looms are zip-tied at the carriage and taped in bundles at the console. It is a working prototype, not a harnessed product: Dupont jumpers are still visible at the end effector, and the console bay is open at the front for probing. That openness is useful while tuning current limits. For a released version, the same pinout should move to a short shielded cable and a closed lid with a strain-relieved exit.

04 The innovation, stated plainly

The innovation is not a new motor or a new board. It is the closed loop between a hand and a three-axis Cartesian arm, with two local sensors in that loop. Teach and jog are physical. Height is measured by time of flight to the ground. A grip is a sponge on the shaft, and it is allowed only when the horizontally mounted IR sensor sees an object. The command path stays on the arm: potentiometer, ADC, dead-zone, step timer, A4988, NEMA 17, plus an I2C range reading and a digital object bit.

05 AprilTag navigation, coordinate design, and custom offset

Navigation on this arm is an AprilTag 36h11, read by S1.5-SRC. The tag is the fiducial. Its detected centre is not yet the board coordinate: the tag sits above the board, so perspective pulls it away from the camera nadir. Trigonometry 1.5 is that correction, plus the fixed cell offset from the tag to the gripper.

The working grid is 20 by 20 by default, square cells, columns A–T and rows 1–20, on a board about 24 inches wide. A cell name such as G11 is column G, row 11. Row 1 is the top of the image. Column A is the left. The gantry runs one axis at a time: all vertical steps first, then all horizontal steps. The serial letters are u, d, l, r, and s for stop.

Heights are measured above the board. The shipped Trigonometry panel values are camera height H = 35.0 in, tag height h = 5.5 in, board height 0. A point at height h and horizontal offset r from the nadir images at the same angle as a flush point at r × H / (H − h). The true board position is the raw pixel pulled back toward the nadir by k = (H − h) / H. With these heights, k = 29.5 / 35 = 0.843. If the camera is not above the board, or the tag is at or above the camera, k falls back to 1 and no correction is applied.

The nadir is the centre of the grid box plus a pixel pivot. Pivot X and pivot Y ship at 0. For a raw tag centre (cx, cy) and nadir (ccx, ccy): corrected = (ccx + k(cx − ccx), ccy + k(cy − ccy)). That corrected point is the tag's cell. There is no second fitted homography; the panel formula is the correction the board uses.

The gripper is not on the tag. The custom offset is a cell delta from the tag cell to the gripper cell, capped at 10 cells on each axis. The shipped offset is 7 cells up and 0 cells right/left (up/down = −7, right/left = 0; up is a negative row step because row 1 is the top). A plan coordinate is the working point. The arm is sent to the tag cell that puts the gripper on that point.

A pre-action nudge trims the last approach. It ships at 0.2 s to the right, armed before pickup and before release, off before keep and press. A contact token from the arm — a whole serial token s, S, or stop — ends a descent. It is not matched as a letter inside another word.

The time-of-flight reading remains the ground distance for the height floor. The IR bit remains the object-present test at the jaws. The AprilTag does not replace either sensor. It names the cell. The offset names where the gripper is relative to that cell.

06 Hardware views

The views below are the HOS Trigonometry 1.5 hardware: the Cartesian stage, the joystick console on the 10 A supply, the Z carriage with the tool motor, and the printed-part family.

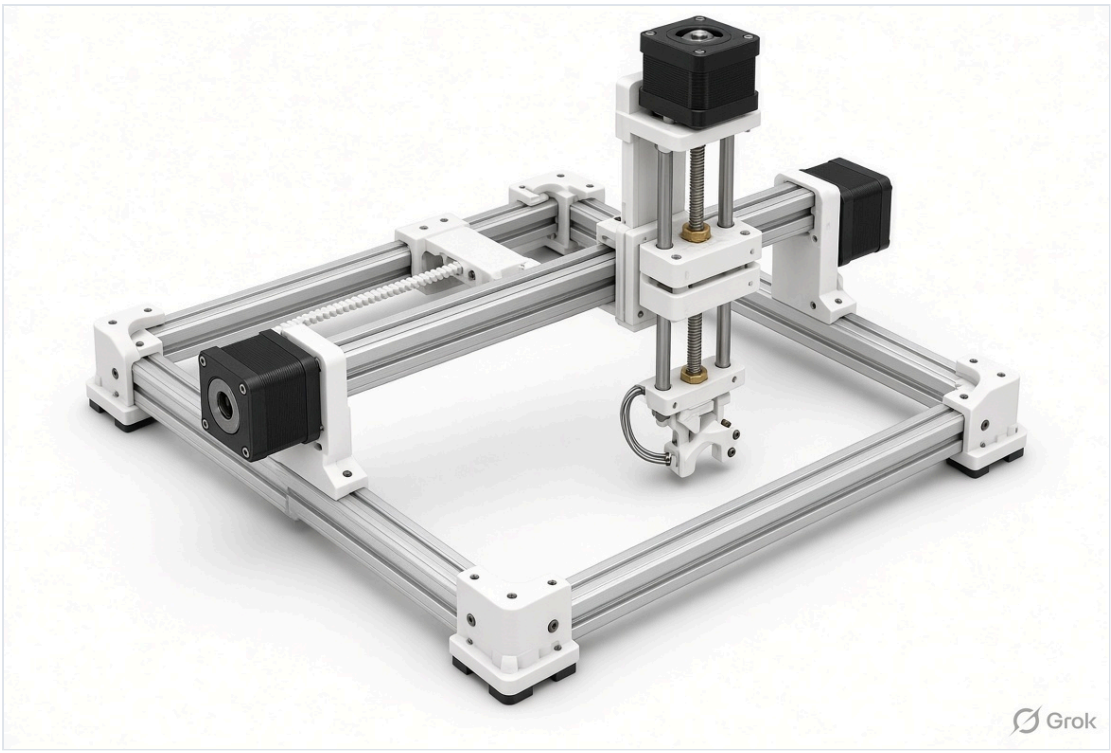


Figure 1. HOS Trigonometry 1.5 stage: belt X/Y, leadscrew Z, NEMA 17s, printed nodes.

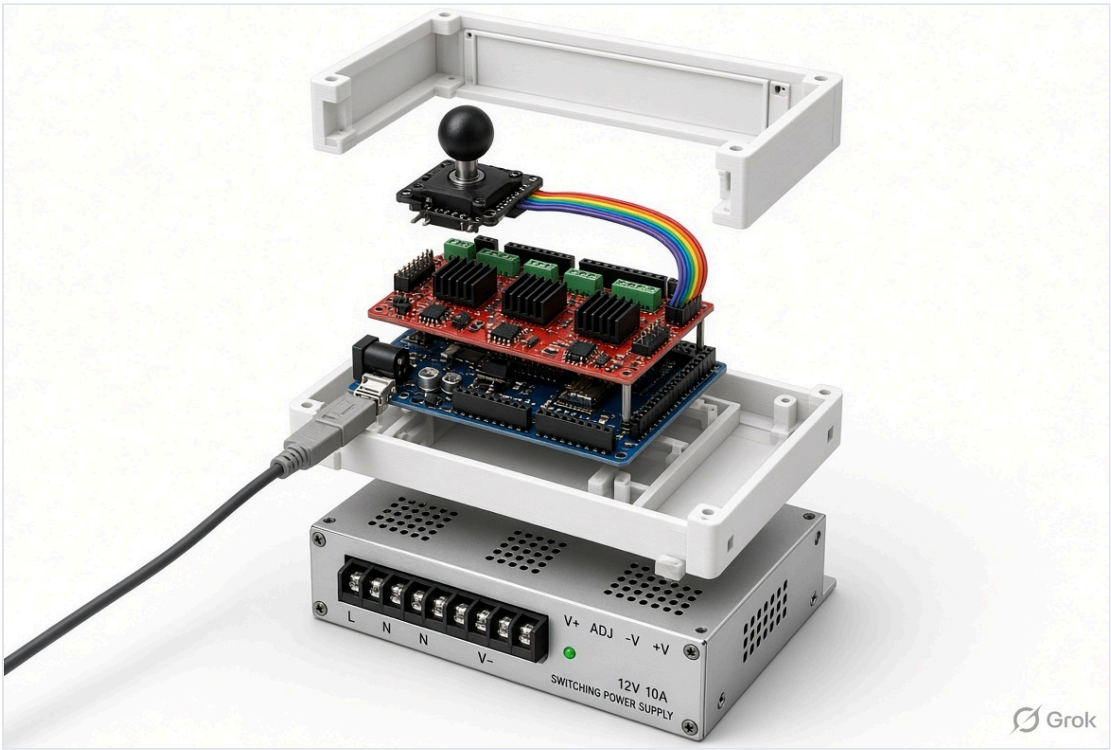


Figure 2. Command console: joystick lid, CNC Shield V3, Arduino Uno, 12 V 10 A supply.

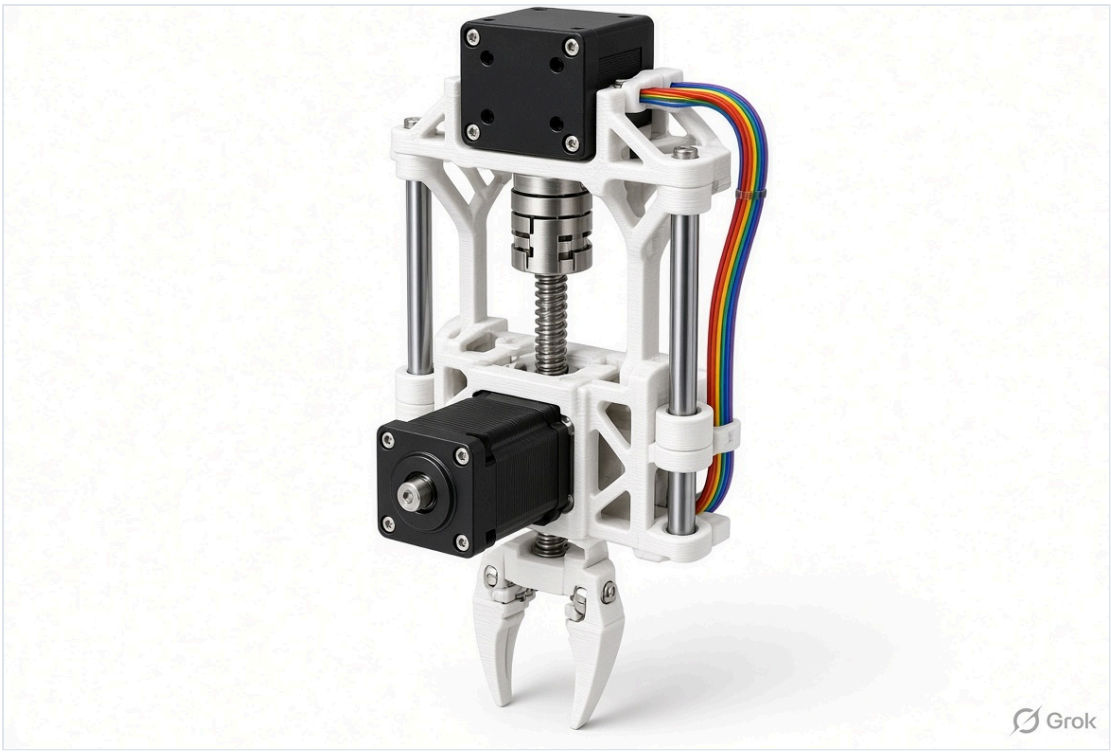


Figure 3. Z carriage: leadscrew motor, coupling, twin rods, tool motor, end effector.

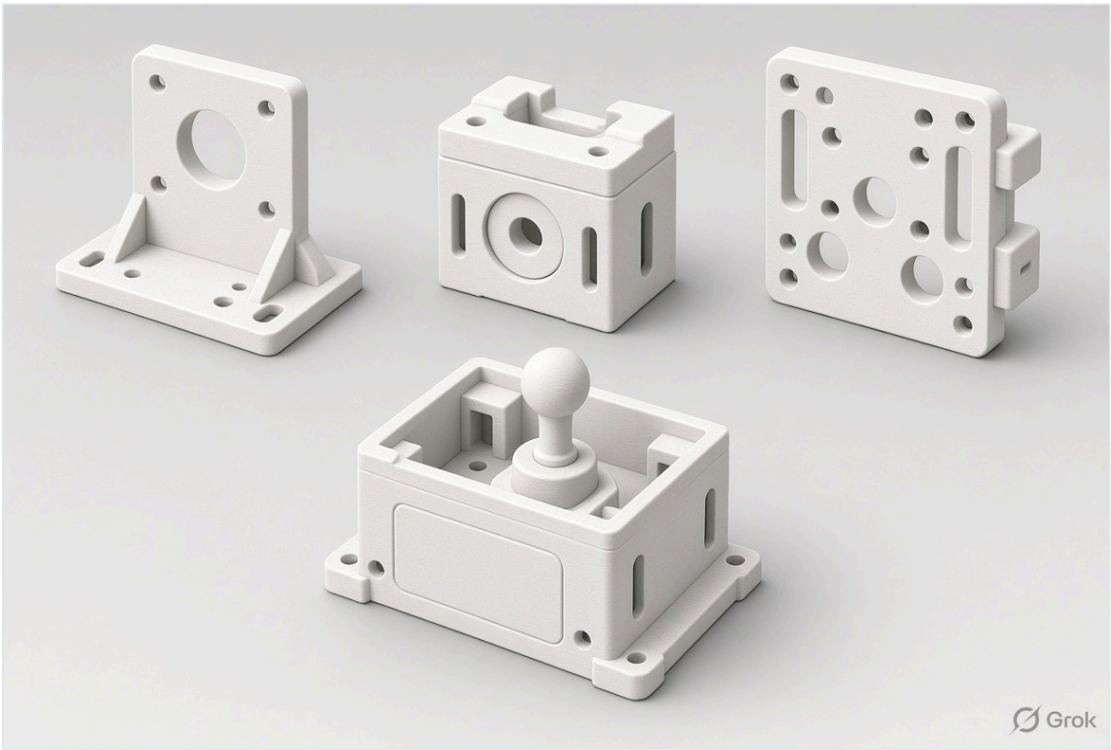


Figure 4. Printed parts: motor foot, idler block, carriage plate, joystick bay.

07 Machine record

The record below is the Trigonometry 1.5 arm: extrusion table, belt span, Z screw and smooth rod, and the joystick console at the corner of the frame.



Figure 5. HOS Trigonometry 1.5 arm. Extrusion table, belt span, Z screw and rod, joystick console.

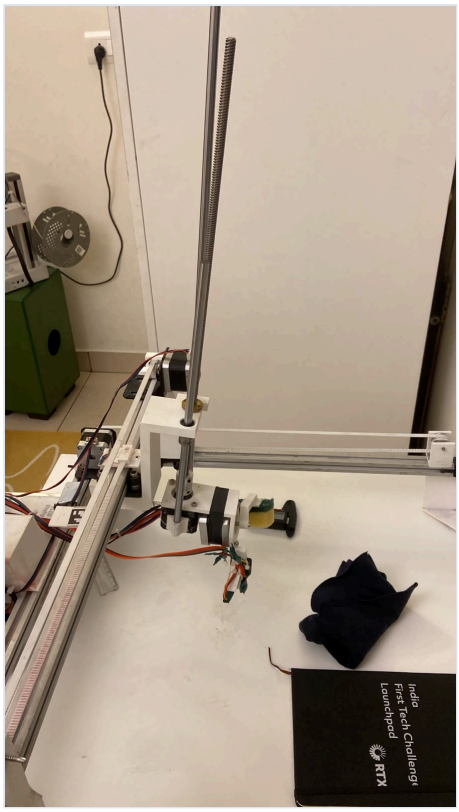


Figure 6. Z upright, leadscrew, carriage with horizontal NEMA 17, end-effector loom.

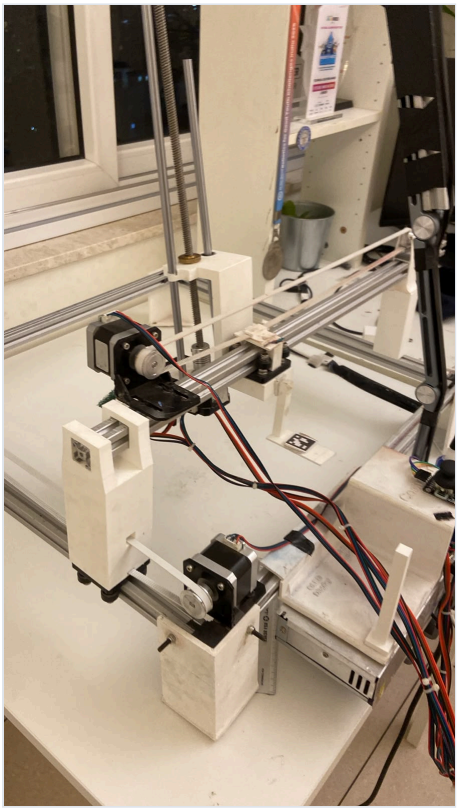


Figure 7. Belt axes. Two NEMA 17s with pulleys, printed feet and idlers, GT2 belt.

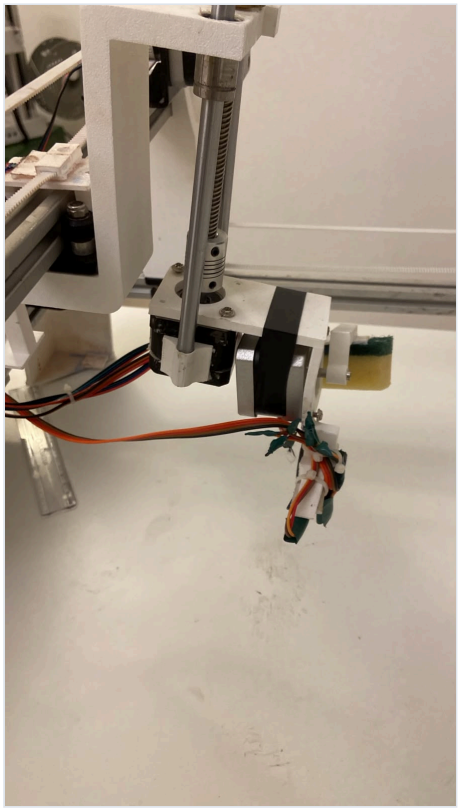


Figure 8. Carriage. Screw coupling, printed clamp, horizontal tool motor, tool loom.

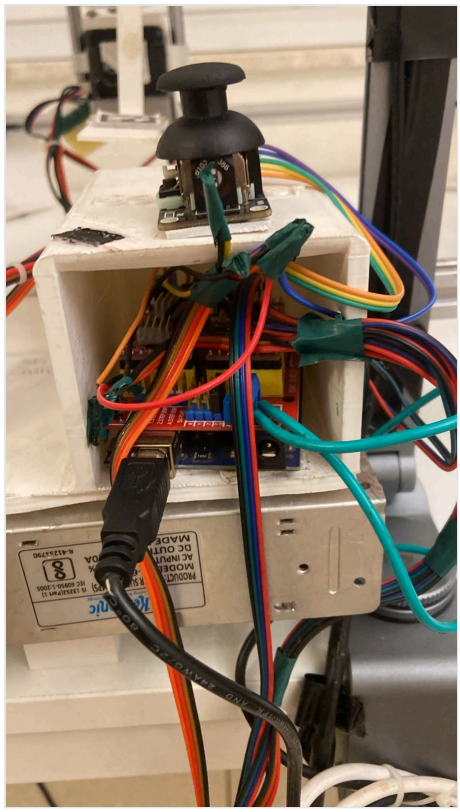


Figure 9. Command console. Joystick on the lid, Uno under the CNC shield, USB, 10 A supply.

08 Printed-part drawings

These are the 3D drawings of the printed parts. The sponge is the grip face: gripper part 2 couples it to the shaft. The IR sensors mount horizontally on gripper part 1, beam across the sponge gap, so an object in the jaws breaks or returns the beam before the sponge closes. The time-of-flight sensor mounts on the same part and faces the ground. The AprilTag connector holds the 36h11 tag used for navigation. The hub holds the Arduino, CNC Shield V3, A4988 drivers, and the supply.

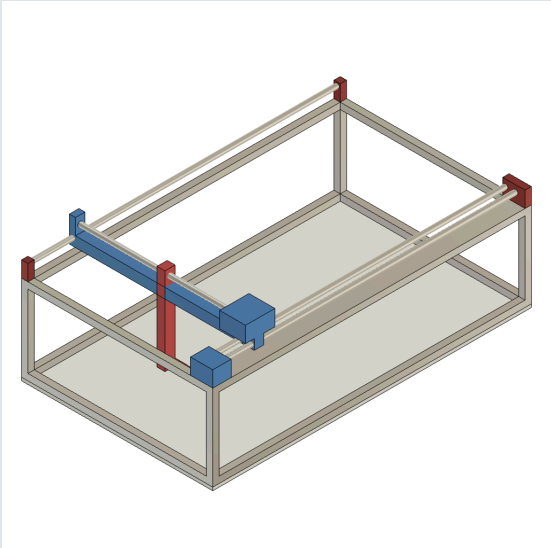


Figure 10. Basic robot drawing. Full Cartesian arm.



Figure 11. Gripper part 1. Joins the Z axis and the gripper stepper, and mounts the horizontal IR sensors and the time-of-flight sensor.

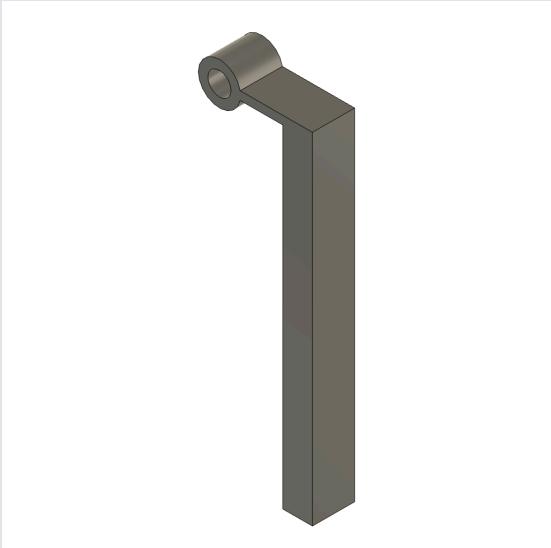


Figure 12. Gripper part 2. Couples the sponge to the shaft.

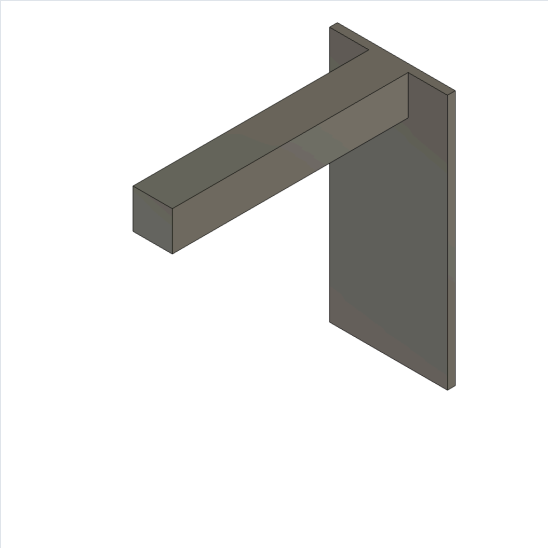


Figure 13. AprilTag connector. Holds the tag used for navigation.

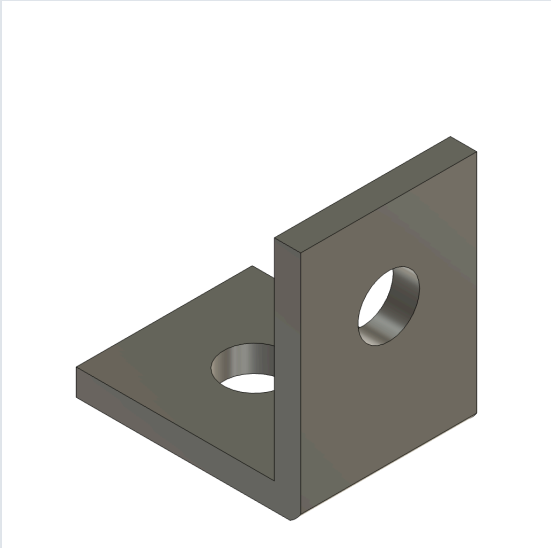


Figure 14. L-bracket. Connects the gripper to the robot.

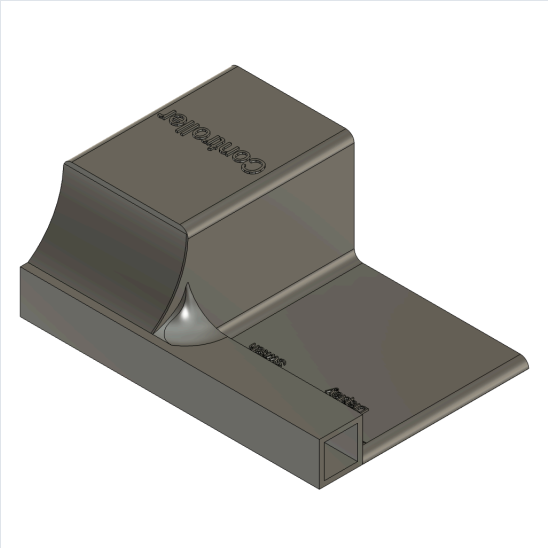


Figure 15. Hub. Holds the controller, supply, Arduino Uno, CNC Shield V3, and A4988 drivers.

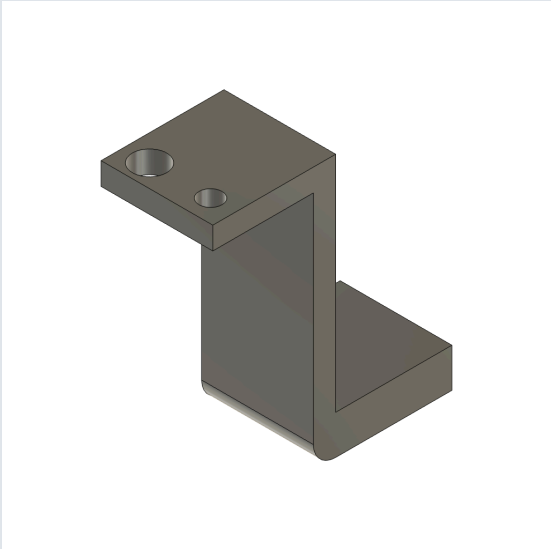


Figure 16. Z-axis mounter. Mounts the Z stepper on the Y axis.

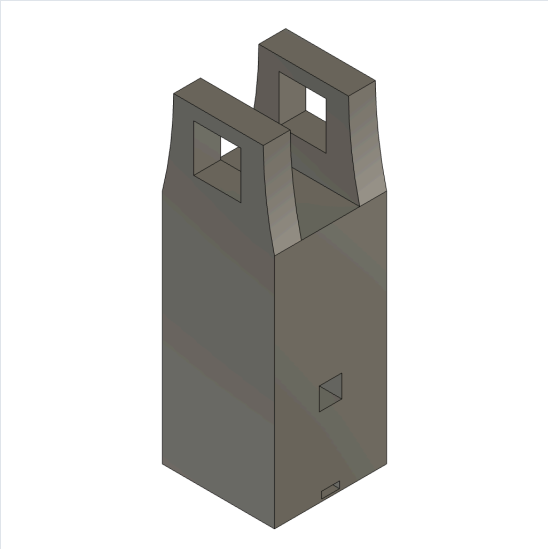


Figure 17. XY inter-axis connector. Joins the Y axis to the X-axis belts and V-slot carriage.



Figure 18. Y-axis holder. Supports the far side of the Y axis.

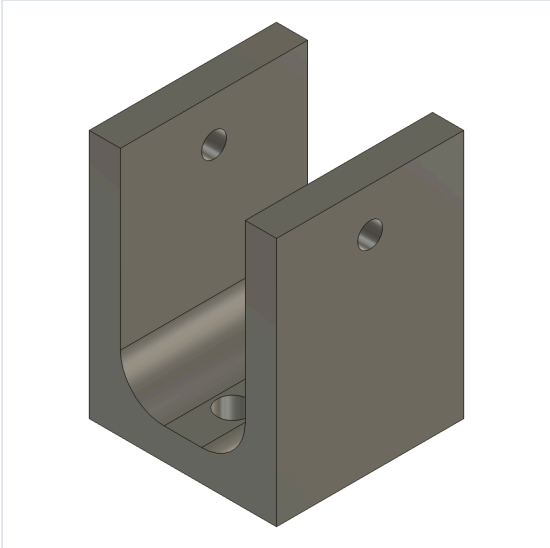


Figure 19. Pulley mount. Holds the pulleys on the axis.



Figure 20. Legs. Hold the robot up from the ground.

09 Bill of materials

Quantities follow the named stack. Confirm before ordering. Third-party lines are not relicensed by this document. The 3MF files for the printed parts are released with this document.

ITEM	QTY	ROLE	STATUS
Arduino Uno (ATmega328P)	1	Command MCU, ADC, step timing	Third-party module, public datasheet
CNC Shield V3	1	STEP/DIR/EN breakout, VMOT, driver sockets	Third-party; open Gerbers exist from several makers
A4988 stepper driver	3 or 4	X, Y, Z, and optional A / tool	Third-party module, Allegro datasheet
Analog joystick module	1	Physical X/Y command and mode switch	Third-party breakout
NEMA 17 stepper, 4-wire	3 or 4	X, Y, Z, optional tool axis	Third-party actuator
Enclosed SMPS, 12 V class, 10 A	1	VMOT only. Label reads 10 A, Keenonic	Third-party. Do not copy the supply internals

2020 V-slot extrusion	frame	Table and gantry rails	Commodity. Cut list is creator design
GT2 belt, 6 mm, and pulleys	2 axes	X and Y transmission	Commodity
T8 leadscrew, nut, coupling	1	Z transmission	Commodity
Smooth rod, 8 mm class, and bearings	2	Z anti-rotation	Commodity
Printed brackets, bay, carriage	set	Creator structure	Original. Release STL/STEP
Time-of-flight sensor	1	Distance from the carriage to the ground	Third-party I2C breakout, VL53L0X class
IR object sensor	1	Mounted horizontally across the sponge gap	Third-party digital IR module
Sponge grip face	1	Compliant jaw on the shaft, via gripper part 2	Consumable
AprilTag 36h11	1	Fiducial for cell navigation and gripper offset	Printed tag, read by S1.5-SRC
USB cable, Dupont, ferrules	lot	Logic, sensor, and motor wiring	Commodity

A 10 A, 12 V supply is enough for three or four NEMA 17s if each A4988 Vref is set for about 1 A or less. Do not run the motors from the Arduino 5 V pin.

10 Electronics and pin map

The shield plugs onto the Uno. A4988 boards sit in the X, Y, Z sockets, and in A if the carriage tool motor is used. Orient each driver so the enable pin matches the shield silkscreen. Set the current-limit potentiometer before the first move: Vref ≈ Imax × 8 × Rsense, with Rsense typically 0.1 ohm on clone boards, so 1 A wants about 0.8 V. Confirm the sense resistor on the module in hand.

SIGNAL	UNO / SHIELD	CONNECTS TO
X STEP / DIR	D2 / D5	A4988 X, then X NEMA 17
Y STEP / DIR	D3 / D6	A4988 Y, then Y NEMA 17
Z STEP / DIR	D4 / D7	A4988 Z, then Z NEMA 17
A STEP / DIR	D12 / D13	Optional tool motor on carriage
Enable (active low)	D8	All A4988 EN, tied on the shield
Joystick VRx	A0	X deflection
Joystick VRy	A1	Y deflection, or Z when the switch is held
Joystick SW	A2	Mode: jog XY / jog Z. INPUT_PULLUP
ToF SDA / SCL	A4 / A5	I2C distance to ground. Keep these off the joystick
IR OUT	D11	Object present at the gripper, active low or high per module
VMOT / GND	Shield power term.	12 V supply. Logic GND common with Uno
5 V / GND	Shield 5 V header	Joystick and IR. ToF breakout from 5 V or 3.3 V as the module requires

Stock GRBL claims D2–D13 and A0–A2. This arm sketch is separate, not a GRBL patch. A4 and A5 stay on I2C for the time-of-flight sensor. Do not put the joystick there. Do not drive STEP from two firmwares at once.

11 Trigonometry 1.5 command layer

This layer accepts two arm commands and nothing else. A grid step `goto_coordinate(column, row)` maps onto this Cartesian arm: X = column pitch, Y = row pitch, Z = tool height. A hand on the stick is the teach input for the same three axes. The two potentiometers are a vector in the arm plane. Magnitude is the hypotenuse, r = sqrt(vx² + vy²), and that magnitude sets the step interval. Direction is the sign of each component. Holding the switch replaces Y with Z, so the arm rises or lowers on the leadscrew. Z motion also watches the time-of-flight reading and will not drive through the height floor. The tool motor closes only if the IR sensor reports an object. The resolver does not drive a mobile base. That resolver is Trigonometry 1.5.

Behaviour as designed for this machine:

- Stick X deflects the X belt. Stick Y deflects the Y belt. Rate grows with deflection, direction follows the sign.
- Switch held: stick Y drives Z on the leadscrew instead of Y. X can stay live or lock, builder's choice; the as-documented default locks X while Z is selected, so a vertical move does not walk the gantry.
- A downward jog stops if the time-of-flight distance to ground is at or under the floor, about 20 mm until calibrated.
- `pickup()` runs the tool motor only when the horizontal IR sensor reads an object in the sponge gap. No object, no close.
- No host is required to jog. USB is for power and for uploading the sketch.

Reference jog loop, released with this document:

```
center = 512; dead = 40; floorMm = 20
vx = analogRead(A0) - center
vy = analogRead(A1) - center
if (abs(vx) < dead) vx = 0
if (abs(vy) < dead) vy = 0
zMode = (digitalRead(A2) == LOW)
groundMm = tof.readRangeMillimeters()
objectThere = (digitalRead(11) == LOW)
if (zMode and vy < 0 and groundMm ⩽ floorMm) vy = 0
if (pickup requested and not objectThere) skip the tool motor
intervalX = map(abs(vx), dead, 512, 4000, 250) // microseconds
on each elapsed interval: DIR pin, then STEP high 5 us, then low
```

Tune the 250 µs floor against the motor. A NEMA 17 on a GT2 belt will skip if the interval collapses too far, especially with the carriage cable drag. Start slower, then raise rate.

12 Mechanical notes from the build

X and Y. Each axis is a NEMA 17, a toothed pulley, a belt, a printed motor foot, and a printed idler or belt clamp. The belt runs along the extrusion. Direction is whatever the pulley winding makes it; swap one coil pair or the DIR invert in firmware if the stick and the table disagree.

Z. A vertical trapezoidal screw, coupling, and brass or POM nut carry the carriage. Two smooth rods stop the nut from spinning. The Z motor is mounted on the moving carriage rather than at the top of a fixed screw — a valid arrangement if the screw is fixed to the upright and the motor-nut unit climbs it. Match the coupling style in Figure 8 before copying the printed clamp.

Tool and sensors. The horizontal motor drives the shaft. Gripper part 2 couples that shaft to a sponge, which is the gripping face: soft contact instead of a hard jaw. The IR sensors are mounted horizontally on gripper part 1 so the beam crosses the sponge gap and answers whether an object is there to grip. The time-of-flight sensor on the same part faces the ground and answers height. Treat the sponge as a consumable. Loom the sensor cables with the tool-motor cable so they cannot reach a pulley.

Console bay. White open box, joystick PCB screwed to the lid, shield inside, USB leaving the front, bay resting on the supply. Leave VMOT wires short and ferruled. Mains stays inside the supply enclosure. This document does not open or modify the supply.

13 Bring-up order

1. Unplug VMOT. Upload the jog sketch. Confirm joystick raw values sit near 512 at rest and swing toward 0 and 1023.
2. Set every A4988 Vref with the motor unplugged. Fit drivers. Common GND between supply and Uno.
3. Connect one motor. Apply 12 V. Jog that axis only. If it chatters, lower Vref or lengthen the step interval. If it runs backward, swap DIR.
4. Repeat for Y, then Z, then the tool motor. Do not enable all four at full current on the first power-up.
5. With the stick released, the table must not creep. Widen the dead zone if it does.
6. Confirm the time-of-flight reading falls as the carriage approaches the ground, and that a downward jog stops at the floor.
7. Confirm the IR bit changes when an object enters the gripper and that pickup() does nothing when the bit says empty.
8. Confirm the AprilTag 36h11 locks to a cell, that k matches the Trigonometry panel, and that a 7-cell-up gripper offset puts the jaws on the named working cell.
9. Add a series emergency switch on the 12 V feed before any unattended move. The joystick switch is a mode control, not a safety rated stop.

Recommended mark, once a UID is issued: "Open Source Hardware, CERN-OHL-S-2.0, certified open source hardware — [UID]". Do not put the OSHWA mark on the machine before certification.

14 Safety and scope

Steppers and the 12 V feed can pinch and can overheat a mis-set driver. Mains wiring stays inside the listed supply. No claim is made of conformity to any product-safety standard. This is prototype documentation of one machine, version 0.1, for study and reproduction. See also Section 22.

15 Certification checklist

Design files and this narrative allow study, modification, making, and sale. Non-commercial and no-derivatives terms are not used.

Original contribution is identified separately from third-party modules that already have public datasheets.

Bill of materials, machine record, pin map, bring-up order, firmware (Shell_1.5.ino), laptop requirements and assembly instructions are included.

Publish the STL or STEP of the printed nodes next to this document. Suggested repo path: Hardware/Trigonometry-1.5 in github.com/Aarav-111/Humanoid-Operating-System.

Suggested certification name: HOS Trigonometry 1.5. Submit only after the repo copy is public. Do not apply the OSHWA mark before a UID is issued.

Documentation licence: CC BY-SA 4.0. Hardware contribution licence: CERN-OHL-S-2.0. Upstream project: github.com/Aarav-111/Humanoid-Operating-System. HOS Trigonometry 1.5.

16 Laptop and system overview

> **A laptop is required to run this project.** The laptop-side Python code runs on the laptop, not on the Arduino. The Arduino Uno runs only the firmware shell.

LAYER	HARDWARE	RUNS	DOES
Laptop / GUI	Laptop or desktop computer + USB webcam	Software/S1.5-SRC.py (Python)	GUI, planning, OpenAI API calls, task assignment, camera capture, AprilTag 36h11 detection, Trigonometry 1.5 coordinate correction
Communication	USB cable, laptop ↔ Arduino Uno	pyserial at **115200 baud**	Laptop decodes each task into movement and sends encoded serial commands (u, d, l, r, s, HU, HD, HX, G0–G90, E, I, J, S)
Hardware execution	Arduino Uno + CNC Shield V3 + A4988 + NEMA 17	Hardware/Firmware/Shell_1.5.ino	Only drives the steppers and gripper, reads joystick, IR and time-of-flight sensors, reports back (e.g. "s", "hx", distance)

The joystick console can jog the arm without the laptop (Section 11), but autonomous operation — any task that uses the camera, the planner or the API — needs the laptop connected and running S1.5-SRC.

16.1 Laptop requirements (from the laptop-side code)

REQUIREMENT	VALUE
Computer / OS	**Apple Mac running macOS.** The code uses AppKit, Foundation and AVFoundation (pyobjc) for its window, menus, camera enumeration (`system_profiler SPCameraDataType`), speech and sound (`afplay`), and runs FastSAM through Core ML. Apple Silicon recommended for Core ML / Neural Engine speed. Windows/Linux are not supported by this version.
Python	Python 3 with tkinter; a project virtual-env at `.venv/` next to the Software folder is auto-used. Exact minimum version: TODO: confirm (OpenCV ≥ 4.7 needed for `cv2.aruco.ArucoDetector`)
Python packages	`opencv-contrib-python numpy pillow openai pyserial sounddevice coremltools pyobjc fonttools certifi`; plus `ultralytics torch` only for the one-time FastSAM build
Vision model	FastSAM-s (`FastSAM-s.pt`), image size 640, converted once to Core ML (`.coreml_cache/FastSAM-s_640.mlmodelc`) by running the script with `--build-fastsam`. Stored in the "Vision AI" folder beside the Software folder or at `HOSX_VISION_AI_DIR`. Runs locally, up to 10 fps by default (4–30 selectable)
Cloud models (internet)	OpenAI API: `gpt-6-luna` (planner, object namer, decisions, error check, dexterity, grip check); `gpt-4o-transcribe` speech-to-text with `whisper-1` fallback
API key	OpenAI API key in environment variable `OPENAI_API_KEY` (paid account; usage billed by OpenAI)
USB	1 port for the Arduino Uno; 1 per external webcam (adapters on USB-C-only Macs)
Camera	Main overhead camera (any USB/UVC webcam, chosen in the GUI by OpenCV index), ~35 in above the board; an optional second "support" camera is supported. Reference frame size used by the shipped calibration: 1089×612 px. Model: TODO: confirm
Microphone / speaker	Laptop built-in, for voice commands and spoken replies (optional)
RAM / storage	TODO: confirm

16.2 Serial protocol (laptop → Arduino)

The laptop finds the Uno automatically by port name (`usbmodem, usbserial, wchusbserial, ttyACM, ttyUSB`) and opens it at **115200 baud**. Commands are ASCII tokens terminated by newline: `u d l r` (move up/down/left/right, one axis at a time), `s`/`S` (stop), `HU HD HX` (Z up, down-until-IR/stop, down with pause),

`G0…G90` (gripper angle), `E` (ToF mm), `I` (IR), `J` (switch state). The Arduino answers with `s` when a descent ends on contact and `hx` on a joystick-down gesture.

17 Complete bill of materials (everything needed to operate)

Supersedes the Section 9 table. **Item 1, the laptop, is required**: without it the Python software, camera vision and API calls cannot run.

#	ITEM	QTY	NOTES
1	**Laptop (required)**	1	Per 16.1. Runs the Python software (not the Arduino)
2	USB webcam + overhead mount	1	Looks down on the board at ~35 in
3	USB cable (USB-A to USB-B, Uno type)	1	Data + logic power laptop → Uno
4	Arduino Uno (ATmega328P)	1	Only microcontroller
5	CNC Shield V3	1	Plugs onto the Uno
6	A4988 stepper driver (with heat sink)	4	X, Y, Z, A (gripper). Heat sinks recommended
7	Microstepping jumpers	9	3 each under X, Y, Z (1/16 step). None under A (full step)
8	NEMA 17 stepper, 4-wire	4	X, Y, Z (leadscrew), A (gripper/tool on carriage)
9	Enclosed SMPS 12 V, 10 A	1	VMOT only. Mains cord + plug for your region
10	Emergency stop switch (series in 12 V feed)	1	Required before unattended moves (Section 13 step 9)
11	Analog joystick module (VRx, VRy, SW)	1	Mounted on console lid
12	VL53L0X time-of-flight breakout (I2C)	1	Adafruit_VL53L0X library
13	Digital IR object sensor module	1	Horizontal across sponge gap
14	Sponge grip face	1	Consumable
15	2020 V-slot extrusion	frame	Lengths/cut list: TODO: confirm
16	V-slot wheel carriages	per axis	X and Y
17	GT2 6 mm belt + GT2 pulleys + idlers	2 axes	X and Y
18	T8 leadscrew, nut, 5–8 mm coupling	1	Z
19	8 mm smooth rod + linear bearings	2	Z anti-rotation
20	3D-printed parts (Hardware/3D Models)	set	basic-robot, gripper-part-1, gripper-part-2-sponge, apriltag-connector, hub, l-bracket, legs, pulley-mount, xy-interaxis-connector, y-axis-holder, z-axis-mounter (.stl). Per-part quantity: TODO: confirm
21	AprilTag 36h11 printout	1	On apriltag-connector
22	Fasteners: M3 (motor), M5 + T-nuts (V-slot)	lot	Standard NEMA 17 = M3; 2020 V-slot = M5. Counts: TODO: confirm
23	Dupont jumpers, motor cables, ferrules, zip ties, 18 AWG for VMOT	lot	Logic, sensor and motor wiring
24	Tools: 3D printer, hex keys, small screwdriver, multimeter, hacksaw	—	Multimeter needed for Vref

18 Wiring (firmware Shell 1.5)

Shell_1.5.ino is the firmware loaded for laptop operation. Its pin use differs from the reference jog loop in Section 10/11; where they differ, follow this table.

SIGNAL	UNO / SHIELD	CONNECTS TO
X / Y / Z STEP, DIR	D2/D5, D3/D6, D4/D7	Shield X, Y, Z driver sockets
A (gripper) STEP / DIR	D12 / D13	Shield A socket — set A-axis clone jumpers to D12/D13
Enable	D8 (active low)	All drivers
IR OUT	D11 (Z+ endstop pin), active LOW	IR sensor
Joystick SW	D9 (X+ endstop pin), pressed = LOW	Joystick switch
Joystick VRx / VRy	A0 (Abort) / A1 (Hold)	VRx = Z jog, VRy = gripper jog
ToF SDA / SCL	A4 / A5	VL53L0X
Sensor power	Shield 5 V / GND	Joystick, IR, ToF
VMOT	Shield 12 V screw terminal	12 V supply via e-stop. Check polarity
USB	Uno USB-B	Laptop

1. **Driver seating:** with all power off, insert each A4988 so its EN pin matches the EN mark on the shield silkscreen (potentiometer away from the motor header on most clones). Fit heat sinks.
2. **Microstepping:** fit all three jumpers (M0–M2) under X, Y, Z for 1/16 step. Leave A without jumpers (full step).
3. **Vref:** Vref = Imax × 8 × Rsense. With 0.1 Ω and ~1 A: ~0.8 V, measured between pot wiper and GND, motor unplugged. Motor rated current: TODO: confirm
4. **Motors:** plug each NEMA 17 into the 4-pin header next to its driver (coil A on 2B/2A, coil B on 1A/1B). Never connect/disconnect a motor while VMOT is on.
5. **Power:** 12 V supply + → e-stop → shield VMOT +; supply − → shield GND. Never feed motors from the Arduino 5 V.
6. **USB:** Uno to laptop. The Uno logic is powered from USB.

19 Mechanical assembly, step by step

1. Print all parts in Hardware/3D Models (material/infill: TODO: confirm).
2. Cut the 2020 V-slot and bolt the table frame together; attach the **legs** at each corner.
3. Mount the X-axis NEMA 17 on its printed foot, fit a GT2 pulley, fit the **pulley-mount** idler at the far end and run the belt along the extrusion; tension and clamp.
4. Join the Y extrusion to the X carriage and belt with the **xy-interaxis-connector**; support the far end with the **y-axis-holder**. Fit the Y motor, pulley and belt the same way.
5. Mount the Z stepper on the Y carriage with the **z-axis-mounter**; fit the T8 leadscrew, coupling and nut, and the two 8 mm smooth rods with bearings. Check the carriage slides without binding.
6. Attach **gripper-part-1** to the Z carriage via the **l-bracket**; mount the horizontal tool (A) NEMA 17 on it. Fit **gripper-part-2-sponge** to its shaft and glue the sponge to it.
7. Mount the IR sensor horizontally on gripper-part-1 across the sponge gap and the VL53L0X facing the ground.
8. Fit the AprilTag 36h11 to the **apriltag-connector** and mount it on the arm, 5.5 in above the board, about 7 cells from the gripper (Section 5).

9. Place the 12 V supply, Uno + CNC shield and drivers into the **hub**; screw the joystick to the lid.
10. Route and zip-tie all cables to the carriage so they cannot reach a pulley; keep VMOT leads short and ferruled.
11. Mount the webcam ~35 in above the 24 in board, centred and looking straight down; lay the 20×20 board under the arm.

20 Software setup

1. **Firmware:** install the Arduino IDE; install the library Adafruit_VL53L0X (Library Manager). Open Hardware/Firmware/Shell_1.5.ino, select board "Arduino Uno" and its port, Upload. Do this with VMOT off.
2. **Laptop (macOS):** install Python 3, then `python3 -m venv .venv && .venv/bin/pip install opencv-contrib-python numpy pillow openai pyserial sounddevice coremltools pyobjc fonttools certifi`; for the one-time model build also `pip install ultralytics torch`, place `FastSAM-s.pt` in "Vision AI", and run the script once with `--build-fastsam`.
3. **API key:** `export OPENAI_API_KEY="sk-..."` (do not paste the key into the source file).
4. **Run:** `python3 Software/S1.5-SRC.py` (or the laptop-side .py supplied; it re-launches itself in `.venv` if present). The software auto-detects the Uno port (names containing usbmodem, usbserial, wchusbserial, ttyACM, ttyUSB) and opens it at **115200 baud**. Close the Arduino Serial Monitor first — only one program can hold the port.
5. Select the overhead camera in the GUI and confirm the AprilTag is detected and locked to a cell.

21 First power-on test

1. Laptop USB only (no 12 V): open Serial Monitor at 115200; send `E` (ToF mm), `I` (IR), `J` (switch) and check sensible replies. Close the monitor.
2. Apply 12 V through the e-stop. Follow the bring-up order in Section 13, one motor at a time.
3. Start S1.5-SRC; send a short manual move (u/d/l/r) then `S`; confirm motion and stop.
4. Test the gripper with `G0`…`G90`, then a full pickup with an object in the gap.
5. Press the e-stop during a move and confirm all motion ends.

Troubleshooting

SYMPTOM	FIX
Software cannot find Arduino	Check USB cable (data, not charge-only), close Serial Monitor, clone Unos with a CH340 chip may need its driver
Motor hums / skips	Lower step rate, re-check Vref, check coil pairs
Axis runs backwards	Reverse motor plug or invert DIR in firmware
Driver hot / shuts down	Lower Vref, add heat sink / fan
ToF reads 0 or fails	Check SDA A4 / SCL A5 and power
Planner errors	Check internet and OPENAI_API_KEY
Tag not detected	Light the board, check focus and camera index, tag must be 36h11

22 Additional safety notes

Keep hands clear of belts, pulleys and the leadscrew while the laptop software is running — the arm can move on a planner decision without a hand on the joystick. Always keep the e-stop within reach. Do not commit your API key to the repository. Unplug the 12 V supply before changing wiring or drivers. Mains wiring stays inside the supply enclosure.